

# Adaptive Graduated Autonomy: Predicting the Rule / Neural-Net / ML Composition for Classification Streams

Anonymous authors

Paper under double-blind review

## Abstract

Production classification sites begin as hand-written rules, are often replaced by a large language model (LLM) call, and may eventually graduate to a trained classical model. Existing practice treats this as a fixed linear ladder that always marches toward a trained model as the destination. We ask a data-centric question instead: **given a data stream’s measurable characteristics, which composition of {rule, LLM, trained ML} is correct, and when?** Across 19 public text classification datasets — scored apples-to-apples on a common test set with four decision-maker tiers (keyword rule, zero-shot LLM, few-shot LLM, classical ML over a training-budget curve) — we find that two early-observable characteristics, **label cardinality** and **rule-accuracy-over-chance**, together with the accumulated labeled-outcome budget, predict the cost-optimal tier (0.62 leave-one-dataset-out, vs 0.51 majority baseline). We turn this into **Adaptive Graduated Autonomy (AGA)**, an algorithm that selects the *terminal* tier per stream and graduates between tiers under a statistical gate, rather than always advancing to the trained model. Against the fixed-ladder baseline, AGA delivers higher mean accuracy (0.836 vs 0.816) at 64% fewer labeled outcomes (1,409 vs 3,934), and keeps the LLM as the terminal decision-maker on 12 of 19 streams — precisely the streams where a trained model never reliably matches it. A tunable non-inferiority margin lets operators trade a bounded accuracy give-up for shedding the LLM’s perpetual per-call cost.

## 1 Introduction

(Adapted from the prior draft’s framing — keep the “every production codebase has classify functions” motivation, drop product/commercial language.)

The contribution is **not** the cascade itself (FrugalGPT-style cost cascades and learning-to-defer route between models already) and **not** the statistical gate (paired McNemar comparison of classifiers is standard). The contribution is **the characterization**: what predicts the right composition, demonstrated across task families and modalities, plus an algorithm and a learned, continually-refined policy that operationalize it.

### 1.1 Contributions

1. An **aligned multi-tier benchmark**: rule / zero-shot LLM / few-shot LLM / classical-ML-curve, all scored on one common test set per dataset, positioned on a shared labeled-outcomes-consumed axis, across 19 text datasets spanning sentiment, intent, moderation, emotion, topic, and spam (plus image and audio modalities via the rule+ML tiers).
2. A **predictive-characteristics result**: label cardinality (Spearman  $\rho=-0.73$  vs outcomes-to-graduate) and rule-over-chance predict both transition depth and the cost-optimal terminal tier.
3. **AGA**, an adaptive phase-gate algorithm that chooses the terminal tier and graduates under a gate, beating the fixed ladder on accuracy *and* labeled-outcome cost.
4. A **non-inferiority (“close-enough”) gate** exposing a single cost-tolerance knob, and a continually-refined meta-policy.

## 2 Setup

- **Tiers / phases.** RULE  $\rightarrow$  MODEL (LLM: zero-shot / few-shot)  $\rightarrow$  ML, mapped to the lifecycle phases. The LLM is a neural-net decision-maker; ML is a classical sklearn head.
- **Datasets (19).** banking77, clinc150, hwu64, atis, snips, trec6, ag\_news, codelangs, sst2, imdb, rotten\_tomatoes, tweet\_eval{offensive,hate,emotion}, dair-ai/emotion, dbpedia14, yahoo\_answers, 20newsgroups, sms\_spam. Modality generalization (image/audio) via CIFAR-10 and ESC-50 on rule+ML.
- **Aligned protocol.** Common 200-example test subset per dataset; rule from a 100-example seed; ML as a curve over training budget; LLM (Claude Haiku) zero-shot and few-shot ( $\leq 40$  exemplars). All tiers placed on the labeled-outcomes-consumed axis (rule $\approx 100$ , zero-shot=0, few-shot=k, ML=budget).

## 3 The composition varies by stream (results)

| dataset           | labels | rule  | ML    | LLM-zs | LLM-fs | winner |
|-------------------|--------|-------|-------|--------|--------|--------|
| clinc150          | 151    | 0.170 | 0.850 | 0.760  | 0.740  | ML     |
| banking77         | 77     | 0.245 | 0.875 | 0.810  | 0.810  | ML     |
| hwu64             | 64     | 0.210 | 0.795 | 0.845  | 0.845  | LLM-zs |
| atis              | 26     | 0.690 | 0.825 | 0.585  | 0.815  | ML     |
| twenty_newsgroups | 20     | 0.135 | 0.670 | 0.665  | 0.695  | LLM-fs |
| dbpedia14         | 14     | 0.605 | 0.970 | 0.975  | 0.990  | LLM-fs |
| codelangs         | 12     | 0.892 | 0.986 | 1.000  | 1.000  | LLM-zs |
| yahoo_answers     | 10     | 0.235 | 0.675 | 0.735  | 0.710  | LLM-zs |
| snips             | 7      | 0.765 | 0.985 | 0.975  | 0.985  | ML     |
| emotion           | 6      | 0.345 | 0.815 | 0.660  | 0.635  | ML     |
| trec6             | 6      | 0.395 | 0.850 | 0.765  | 0.880  | LLM-fs |
| ag_news           | 4      | 0.340 | 0.895 | 0.850  | 0.910  | LLM-fs |
| tweet_emotion     | 4      | 0.385 | 0.620 | 0.875  | 0.865  | LLM-zs |
| imdb              | 2      | 0.515 | 0.895 | 0.935  | 0.975  | LLM-fs |
| rotten_tomatoes   | 2      | 0.500 | 0.720 | 0.935  | 0.945  | LLM-fs |
| sms_spam          | 2      | 0.935 | 0.955 | 0.905  | 0.985  | LLM-fs |
| sst2              | 2      | 0.515 | 0.780 | 0.965  | 0.975  | LLM-fs |
| tweet_hate        | 2      | 0.490 | 0.565 | 0.805  | 0.675  | LLM-zs |
| tweet_offensive   | 2      | 0.680 | 0.775 | 0.755  | 0.725  | ML     |

Key reading: classical ML wins on high-cardinality intent (banking77 0.875 vs LLM 0.81); the LLM wins decisively on low-cardinality sentiment/moderation (sst2 0.78 $\rightarrow$ 0.975, imdb 0.895 $\rightarrow$ 0.975); few-shot  $\geq$  zero-shot almost everywhere.

## 4 Adaptive Graduated Autonomy

- **oracle\_terminal** (cost-aware): highest-accuracy tier; among those within  $\epsilon$ , prefer cheapest to operate. The LLM wins only when strictly  $>\epsilon$  better.
- **Tie-to-graduate and crossover:** ML graduates once it *ties* (non-inferiority) the LLM, not once it beats it — because trained ML is far cheaper per call.
- **CostToleranceGate:** the non-inferiority gate; the  $\epsilon$  knob is operator-tunable.
- **AGAMetaPolicy:** predicts the terminal tier from early-observable characteristics; continually re-fined as new streams append rows.

### 4.1 AGA vs the fixed ladder

Mean accuracy 0.836 vs 0.816; mean labeled outcomes 1,409 vs 3,934 (−64%); LLM kept as terminal on 12/19.

| dataset           | oracle         | AGA(LODO)      | AGA_acc | fixed_acc | AGA_cost | fix_cost |
|-------------------|----------------|----------------|---------|-----------|----------|----------|
| ag_news           | ml             | ml             | 0.895   | 0.895     | 5000     | 5000     |
| atis              | ml             | ml             | 0.825   | 0.825     | 2000     | 2000     |
| banking77         | ml             | model_zeroshot | 0.810   | 0.875     | 0        | 5000     |
| clinc150          | ml             | model_fewshot  | 0.740   | 0.850     | 40       | 5000     |
| codelangs         | ml             | model_fewshot  | 1.000   | 0.986     | 36       | 500      |
| dbpedia14         | ml             | ml             | 0.970   | 0.970     | 2000     | 2000     |
| emotion           | ml             | model_zeroshot | 0.660   | 0.815     | 0        | 5000     |
| hwu64             | model_zeroshot | ml             | 0.795   | 0.795     | 5000     | 5000     |
| imdb              | model_fewshot  | model_fewshot  | 0.975   | 0.895     | 40       | 5000     |
| rotten_tomatoes   | model_fewshot  | model_zeroshot | 0.935   | 0.720     | 0        | 5000     |
| sms_spam          | model_fewshot  | ml             | 0.955   | 0.955     | 2000     | 2000     |
| snips             | ml             | ml             | 0.985   | 0.985     | 500      | 500      |
| sst2              | model_fewshot  | model_fewshot  | 0.975   | 0.780     | 40       | 5000     |
| trec6             | model_fewshot  | model_zeroshot | 0.765   | 0.850     | 0        | 5000     |
| tweet_emotion     | model_zeroshot | model_fewshot  | 0.865   | 0.620     | 40       | 2000     |
| tweet_hate        | model_zeroshot | model_fewshot  | 0.675   | 0.565     | 40       | 250      |
| tweet_offensive   | ml             | model_fewshot  | 0.725   | 0.775     | 40       | 500      |
| twenty_newsgroups | model_fewshot  | model_zeroshot | 0.665   | 0.670     | 0        | 10000    |
| yahoo_answers     | model_zeroshot | ml             | 0.675   | 0.675     | 10000    | 10000    |

LODO oracle-match: 6/19 = 0.32

Mean accuracy AGA 0.836 vs fixed-ladder(ML\_PRIMARY) 0.816

Mean labeled outcomes AGA 1409 vs fixed-ladder 3934

AGA keeps LLM as terminal (skips ML training): 12/19

AGA picks rule/ML over not-better LLM (avoids perpetual cost): 4/19

## 4.2 Meta-policy generalization

Four-tier leave-one-dataset-out: 0.623 (vs 0.51 majority). Drivers: log label-cardinality (0.42), rule-over-chance (0.26), outcome budget (0.18). [Feature-importance figure.]

## 5 Limitations (honest)

- Early-only terminal prediction is weak (0.32 oracle-match); the budget-aware policy is stronger (0.62). Oracle noisy at n=200 — larger-n re-run planned.
- MODEL tier is text-only so far; audio/video MODEL requires a multimodal LLM (the gate is modality-agnostic; only the LLM input differs).
- Single LLM (Haiku); single classical head family.

## 6 Related work

FrugalGPT / LLM cascades (cost routing); learning-to-defer (per-example deferral); champion-challenger + shadow deployment (MLOps); McNemar / Dietterich (classifier comparison). AGA differs: a *learned, continually-refined, cross-stream* policy that predicts the right composition from data-stream characteristics and chooses the terminal tier, rather than routing per example or hand-tuning a cascade.

## 7 Reproducibility

All loaders, the aligned harness, AGA, the gate, and the meta-classifier are released; every number regenerates from `scripts/aligned_tier_eval.py`, `scripts/evaluate_aga.py`, `scripts/build_meta_classifier.py`, and `scripts/analyze_dmlr_characteristics.py`.